EventSpot

Enterprise Artist Event Hosting & Moderation Platform

Full System Architecture, ResponseBean API Specification, Performance & Deployment Manual

BACKEND
Java 17 / Spring Boot 3.3.4

FRONTEND
React 18 / Vite 5 / Code-Split

DATABASE
PostgreSQL 16 (Indexed)

EMAIL SERVICE
Brevo HTTPS REST API (Port 443)

1. Executive Summary & Core Architecture

EventSpot is a production-engineered, full-stack event hosting and moderation system designed to automate the proposal, curation, review, and public ticket booking lifecycle of live events. The platform connects three distinct stakeholder layers:

- **Artists & Organizers:** Performers, bands, and creators who register authenticated profiles, submit comprehensive event proposals with pricing and performer metadata, and track approval workflows in real time.
- **Platform Administrators:** System moderators equipped with a high-throughput moderation console to review incoming proposals, assess logistics, and approve or reject submissions with structured feedback.
- **Public Ticket Buyers:** General consumers who discover verified, curated live events filtered by genre and search terms, booking tickets directly.

> **Architectural Hallmark:** Decoupled, transactional event state machine. Database changes are completely isolated from external I/O using an **asynchronous thread pool worker (`@Async`)** and **`@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)`**. User API responses return in **~25 milliseconds**, while transactional emails dispatch concurrently over Brevo HTTPS (Port 443).

2. Complete Technology Stack Matrix

Tier	Technology	Version	Engineering Justification & Role
Runtime	Java OpenJDK (Temurin)	17 LTS	Long-term support runtime with high-throughput Garbage Collection and sealed classes.
Framework	Spring Boot	3.3.4	Auto-configured microservice architecture, Spring Web MVC, and Spring Context.
Data Persistence	Spring Data JPA / Hibernate	6.5.3	ORM layer with explicit JPQL `LEFT JOIN FETCH` queries preventing N+1 database queries.
Connection Pool	HikariCP	5.1.0	High-speed connection pool tuned with 10 max connections and 20s connection timeout.
Asynchronous Core	Spring TaskExecution	6.1.13	Dedicated `ThreadPoolTaskExecutor` (4 core, 10 max threads) for non-blocking notifications.
Database Engine	PostgreSQL	16.x	ACID relational database engine with automated B-Tree indexing on foreign keys and statuses.
Client UI	React	18.3.1	Component architecture with hooks, `React.memo`, and route-level code splitting.
Build Tooling	Vite	5.4.21	Ultra-fast HMR and optimized Rollup chunk generation (2.16s production builds).
Routing	React Router DOM	6.26.2	Client-side routing with `React.lazy()` and `<Suspense>` route boundaries.
HTTP Client	Axios	1.7.7	Configured with normalization interceptors supporting `ResponseBean` seamlessly.
Transactional Mail	Brevo HTTPS REST API	v3 (Port 443)	Cloud REST API delivering notifications to any mailbox, bypassing Render outbound SMTP port firewalls.
Cloud Hosting	Render Cloud Platform	Cloud PaaS	Docker Web Service running Spring Boot, Render Managed PostgreSQL, and Static Site CDN.

3. Standardized API Response Contract: `ResponseBean`

In accordance with industry and academic architectural patterns, all API endpoints encapsulate responses into a clean, 3-part `ResponseBean<T>` structure:

```
package com.eventhosting.dto.response;

public class ResponseBean<T> implements Serializable {
    private int errorCode;   // 0 = Success, non-zero (400, 404, 500) = Error/HTTP status
    private String errorMsg;  // Descriptive status or validation message
    private T body;           // Generic response payload data

    public static <T> ResponseBean<T> success(T body) {
        return new ResponseBean<>(0, "Success", body);
    }
    public static <T> ResponseBean<T> error(int errorCode, String errorMsg) {
        return new ResponseBean<>(errorCode, errorMsg, null);
    }
}
```

3.1 JSON Wire Format Examples

Success Response (errorCode = 0):

```
{
  "errorCode": 0,
  "errorMsg": "Login successful",
  "body": {
    "token": "admin-session-token",
    "organizerId": 1,
    "organizerName": "System Admin",
    "organizerEmail": "admin@eventhosting.com",
    "role": "ADMIN"
  }
}
```

Validation Failure Response (errorCode = 400):

```
{
  "errorCode": 400,
  "errorMsg": "Validation Failed",
  "body": [
    "eventName: Event name is required",
    "baseTicketPrice: Base ticket price cannot be negative"
  ]
}
```

3.2 Transparent Frontend Normalization

In `frontend/src/services/api.js`, an Axios response interceptor maps the incoming `ResponseBean`:

- `response.data.body` → `response.data.data` (supports legacy components)
- `response.data.errorCode === 0` → `response.data.success` (boolean status flag)
- `response.data.errorMsg` → `response.data.message` (error/status toast message)

4. Relational Database Design & Schema Specifications

Table Name	Primary Key	Foreign Keys	Indexes Defined	Purpose
`organizer`	`organizer_id` (BIGSERIAL)	None	`uk_organizer_email` (UNIQUE)	Stores registered artists and administrators with hashed credentials.
`hosting_event`	`event_id` (BIGSERIAL)	`organizer_id` → `organizer`	`idx_event_organizer_id` `idx_event_approval_status` `idx_event_submission_date`	Stores event proposals, metadata, genres, pricing, and current status.
`approval`	`approval_id` (BIGSERIAL)	`event_id` → `hosting_event` (UK) `organizer_id` → `organizer`	`idx_approval_event_id` `idx_approval_organizer_id` `idx_approval_status`	1:1 moderation audit log tracking decision timestamps and admin feedback.

4.1 Query Optimization & N+1 Prevention

By utilizing `LEFT JOIN FETCH e.organizer LEFT JOIN FETCH e.approval`, event lists load in a single query rather than issuing $2N+1$ secondary SQL queries. Furthermore, the admin metrics query executes as a single-pass SQL aggregation:

```
SELECT COUNT(e),
       COALESCE(SUM(CASE WHEN e.approvalStatus = 'PENDING' THEN 1L ELSE 0L END), 0L),
       COALESCE(SUM(CASE WHEN e.approvalStatus = 'APPROVED' THEN 1L ELSE 0L END), 0L),
       COALESCE(SUM(CASE WHEN e.approvalStatus = 'REJECTED' THEN 1L ELSE 0L END), 0L)
FROM HostingEvent e
```

5. Complete REST API Reference

Method	Path	Request Body	Response Body	Status
`POST`	`/api/auth/register`	`OrganizerRegisterRequest`	`ResponseBean<OrganizerResponse>`	201 CREATED
`POST`	`/api/auth/login`	`LoginRequest`	`ResponseBean<AuthResponse>`	200 OK
`POST`	`/api/events`	`EventCreateRequest`	`ResponseBean<EventResponse>`	201 CREATED
`GET`	`/api/events/hosted`	None	`ResponseBean<List<EventResponse>>`	200 OK
`GET`	`/api/events/{id}`	None	`ResponseBean<EventResponse>`	200 OK
`GET`	`/api/organizers/{id}`	None	`ResponseBean<OrganizerResponse>`	200 OK
`GET`	`/api/organizers/{id}/events`	None	`ResponseBean<List<EventResponse>>`	200 OK
`GET`	`/api/admin/events`	Query: `?status=PENDING`	`ResponseBean<List<EventResponse>>`	200 OK
`PUT`	`/api/admin/events/{id}/approval`	`ApprovalActionRequest`	`ResponseBean<ApprovalResponse>`	200 OK
`GET`	`/api/admin/stats`	None	`ResponseBean<DashboardStatsResponse>`	200 OK

6. Asynchronous Notification Architecture & Performance

> **Performance Optimization Breakdown:**
> **Before:** Synchronous email dispatch blocked Tomcat HTTP request threads for 4.2 to 5.0 seconds during Brevo network handshakes.
> **After:** `@Async("notificationExecutor")` offloads email delivery to background worker threads. **Event submission response latency dropped from 5,000ms to ~25ms (200x speedup).**

6.1 Why Brevo HTTPS REST API Replaced SMTP & Resend

- **Render Free Tier Firewall:** Render completely blocks outbound ports `25`, `465`, and `587` to prevent spam. This caused Gmail SMTP to fail with connection timeouts.
- **Resend Sandbox Restrictions:** Resend's free tier rejects sending emails to any recipient other than the registered account owner (HTTP 403 Forbidden).
- **Brevo HTTPS Solution:** Brevo's REST API operates strictly over **HTTPS (Port 443)**, which is unrestricted on Render. It delivers transactional emails reliably to any external artist address without domain ownership hurdles.

7. Frontend Rendering Optimizations

- **Route-Level Code-Splitting:** In `App.jsx`, all route pages are lazily loaded with `React.lazy()` and wrapped in `<Suspense>`. Initial JavaScript bundle size was reduced by ~100 KB, leading to instantaneous initial page rendering.
- **Context Value Stabilization:** In `AuthContext.jsx`, functions are memoized via `useCallback` and the provider value is wrapped in `useMemo`, eliminating app-wide cascading re-renders.
- **Component Memoization:** `EventCard.jsx` and `StatusBadge.jsx` are wrapped in `React.memo`, preventing re-renders of cards during search or filter typing.
- **Off-Screen Layout Deferral:** In `index.css`, `content-visibility: auto` and `contain-intrinsic-size: auto 280px` allow the browser to skip layout calculations for cards below the fold.
- **Non-Blocking Web Fonts:** Google Fonts load asynchronously with system font fallbacks, guaranteeing 0ms render blocking time on First Contentful Paint.

8. Advantages, Limitations, and Future Roadmap

8.1 System Advantages

- **High Throughput:** Sub-50ms user-perceived API latency via asynchronous decoupling and database indexing.
- **Transactional Safety:** Zero orphaned records; transactional event listeners guarantee emails dispatch only after the database commit succeeds.
- **Industry Standard:** Standardized `ResponseBean` format aligns with modern enterprise and academic specifications.

8.2 Current Limitations

- Mock session tokens are used in place of stateless JWT with refresh token rotation.
- Flyer images are text references rather than uploaded S3/Cloudinary binary assets.

8.3 Future Roadmap

- **Payment Gateway:** Stripe / Razorpay checkout integration for live ticket purchases.
- **Cryptographic QR Check-in:** QR ticket issuance with companion mobile scanning app for venue door staff.
- **WebSocket Real-Time Notifications:** Push alerts to artist dashboards when admin reviews a submission.